

Uncertainty-Aware Action Selection for LLM Agents

Using Conformal Prediction and BFS-Based Planning



M.Tech Thesis Presentation

Presenter

Tiku Mohammd

Roll No: 24CS60R81

Supervisor

Prof. Sourangshu Bhattacharya

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

April 2026

Motivation

The Challenge of LLM Agents

- LLM agents operate in **sequential decision spaces**, where each action affects future states
- Action generation is **inherently stochastic**, causing **irreversible trajectory failures**
- Early incorrect decisions **cascade**, making recovery difficult in long-horizon tasks

Limitations of Existing Solutions

ReAct, ReAct++

- Treat all sampled actions **symmetrically**
- Require **multiple expensive** LLM calls
- Lack **action-level prioritization**

Key Gap: No principled **uncertainty quantification** for action selection in agentic LLMs

Can we make LLM agents reliable and cost-efficient by selecting actions based on calibrated uncertainty?

Problem Statement

Given a state s_t in an interactive environment, an LLM generates a set of candidate actions.

Objective: The objective is to **select an action (or set of actions)** that leads to successful task completion

Key Challenges:



Uncertainty

LLM outputs are stochastic and uncalibrated



Error Propagation

early mistakes lead to irreversible failures



Inefficiency

repeated sampling (self-consistency) is costly

Key Question

How can we select actions with statistical confidence while maintaining efficiency in sequential decision-making?

Objective:

Select high-quality actions under uncertainty while minimizing LLM calls

Limitations of Existing Methods (ReAct / ReAct++)

ReAct / ReAct++ Behavior

- Generates multiple candidate actions
- Uses majority voting / similarity
- Outputs a **single final action**

Key Issues

- Collapses diverse candidates into one decision
- No notion of **confidence or uncertainty**
- Heuristic aggregation (no guarantees)
- Requires multiple expensive LLM calls

No principled framework for uncertainty-aware decision making

Key Idea: From Action Selection to Uncertainty-Aware Planning

Existing Methods

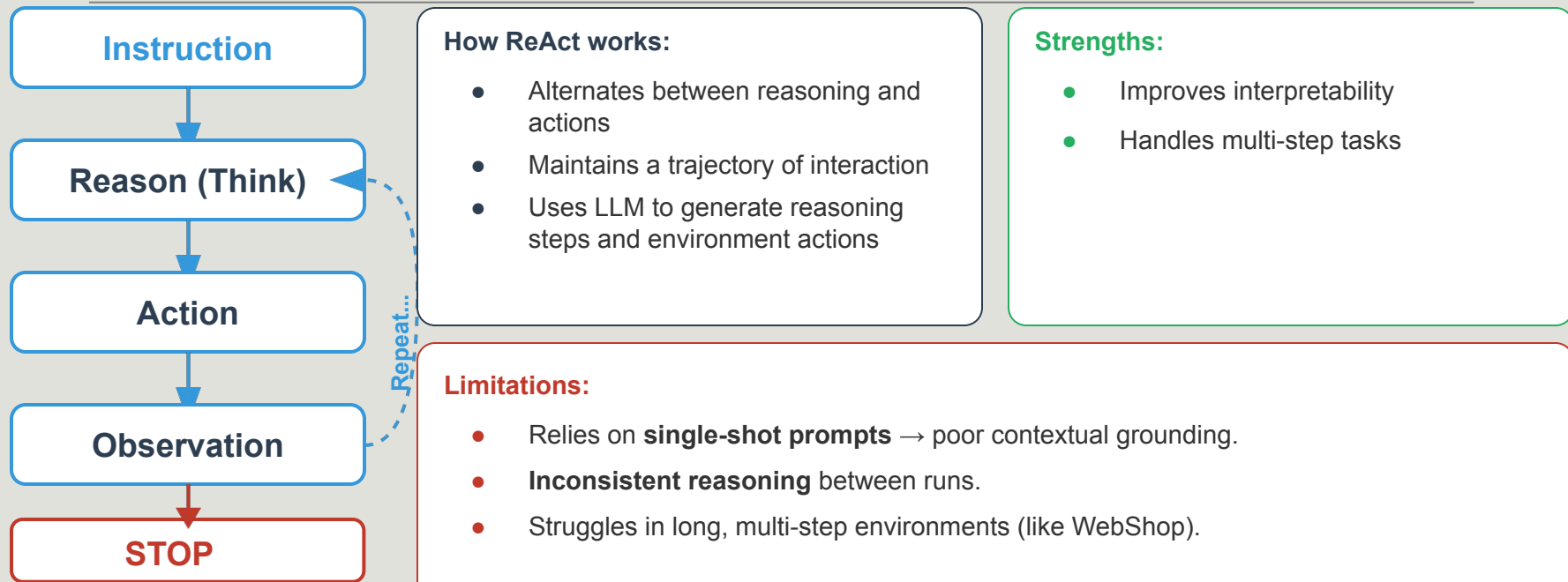
- Many candidate actions
- Heuristic aggregation
- Output → **Single action**

Proposed Method

- Learn action quality (scoring)
- Construct **action set** (conformal)
- Explore via **BFS planning**

Select a set, not a single action

Background: The ReAct Framework



Still relies on single-action decisions at each step

The WebShop Environment

Platform Overview

- A large-scale simulated **e-commerce platform** designed for grounded language interaction.

Key Features:

- 1.1M+** real-world products
- 12,000** human-written instructions
- Structured product attributes
- Multi-step reasoning tasks
- Attribute-based reward system

WebShop search

Instruction:
i'm looking for a small portable folding desk that is already fully assembled; it should have a khaki wood finish, and price lower than 140.00 dollars

portable folding desk khaki wood **1** Search

2 results

Back to Search
Page 1 (Total results: 50)
Next >

item-detail

Description: Product: laptop desk. Product material: walnut. Product weight: 4.6pounds. Material: high quality thick steel pipe, black brushed sheet. Special design: black brushed smooth table top, increase the length and width of the table, it is possible to place the computer and various items. Function: Can be used as computer desk, dining table, bedside table. Product size: 23.6x15.7x11 inches

item-detail

- [Large Size]** Holds laptops up to 17 inches. It also have spacious space (23.6x15.7x11 inches) for your laptop, notebook, mouse, pen and coffee. Its generous size gives this versatile desk even more flexibility.
- [Wide Application]** Our foldable lap desk can be used as a

item

MENHG Folding Laptop Table Bed Desk PC Lap Desk with Drawer Book Stand Reading Holder Leg Space Laptop Bed Tray Foldable Lazy Table Breakfast Desk Sofa Small Desk for Small Space

Price: \$100.0
Rating: N.A.

Description Overview

4.1 4.2

Color
black khaki white **3**

Buy Now 5

Reward: 1.0

Example of a trajectory that falls into a loop

Action: reset
Observation:
WebShop
Instruction:
i'm looking for hair treatments that are sulfate and paraben free and are of high quality too. i need it in bottle for with 60 capsules, and price lower than 60.00 dollars
[Search]

Action: search[sulfate free hair treatment paraben free high quality 60 capsules]
Observation:

[Back to Search]
Page 1 (Total results: 50)
[Next >]
[B07MTPW3Z7]
Biotin Shampoo For Hair Growth With DHT Blockers, Sulfate-Free, Paraben-Free, Thickening Shampoo, For Thicker and Fuller Hair, Hair Loss Shampoo, For Men and Women (16 oz)
\$16.95
[B095Z3JB69]
Act+Acre Restorative Hair Mask Hair Treatment Mask with Castor Oil (4.5 fl oz), Nourishing Sulfate Free Shampoo (10 Fl Oz), Conditioner (10 Fl Oz), and Scalp Oil Treatment (3 Fl Oz)
\$100.0
[B07GSVD92H]
OGX Extra Strength Damage Remedy + Coconut Miracle Oil Hair Mask, Extra Hydrating & Softening Anti-Frizz Treatment to Help Repair Hair, Paraben-Free, Sulfated-Surfactants Free, 6 oz
\$6.99

Action: think[B07MTPW3Z7 is a shampoo, not a hair treatment. B095Z3JB69 is too expensive. B07GSVD92H is a hair mask, not a treatment. I need to search again.]
Observation: OK.

Action: search[sulfate free hair treatment paraben free high quality]
Observation: Invalid action!

Action: search[sulfate free hair treatment paraben free high quality hair treatment]
Observation: Invalid action!

Action: search[sulfate free hair treatment paraben free high quality hair treatment bottle]
Observation: Invalid action!

And so on...
Code will terminate after 15 iterations.

An example illustrating an incorrect decision made by an LLM

Action: reset

Observation:

WebShop

Instruction:

i need gluten free vegetarian smoked peppered bacon - 4 ounce (pack of 2), and price lower than 50.00 dollars

[Search]

Action: search[gluten free vegetarian smoked peppered bacon 4 ounce pack of 2]

Observation:

[Back to Search]

Page 1 (Total results: 50)

[Next >]

[B06Y96MXJV]

Smoked Bacon Sea Salt 3-Pack: Smoked Bacon Chipotle, Smoked Bacon and Onion, Smoked Peppered Bacon - All-Natural Sea Salts Slowly Smoked Over Alderwood - No Gluten, No MSG, Non-GMO (12 total oz.)

\$29.99

[B06Y96N1KG]

Spicy Hot Pepper Sea Salt 3-Pack: Ghost Pepper, Jalapeno, Habanero - All Natural, Delicious Blends of Peppers & Sea Salt - Gluten-Free, Kosher, No MSG, Non-GMO (12 total oz.)

\$29.99

[B07GJTKYJQ]

Louisville Vegan Jerky - 5 Flavor Variety Pack, Non-GMO Soy Protein, Gluten-Free (Black Pepper, Buffalo Dill, Pepperoni, Maple Bacon, and Carolina BBQ, 3 Ounces)

\$42.99

Action: think[The item has options 'Smoked Peppered Bacon' and '4 ounce (pack of 2)' and seems good to buy.]

Observation: OK.

Action: click[Smoked Peppered Bacon]

Observation: Invalid action!

Action: click[4 ounce (pack of 2)]

Observation: Invalid action!

Action: think[The item has options 'Smoked Peppered Bacon' and '4 ounce (pack of 2)' and seems good to buy.]

Observation: OK.

Action: click[Smoked Peppered Bacon]
Observation: Invalid action!

Action: click[4 ounce (pack of 2)]
Observation: Invalid action!

Action: click[B06Y96MXJV]
Observation:
[Back to Search]
[< Prev]

flavor [cocktail salts][dessert toppers][exotic][finishing salts][foodie gift][french blends][french classics][fruits & flowers][garlic lover's][gourmet grillmaster][hawaiian][herb garden][i love you & food][italian classics][italian flavors][keto smoked][mediterranean diet][natural salts][olive oil dippers][pacific cooking][paleo][pegan smoked][pizza salts][smoked #1][smoked bacon][smoked classics][smoked grillmaster][spicy chili peppers][steak rubs][summer fruits][taco seasonings][the pepper][truffle][tuxedo][vegan smoked][vegetarian smoked]
size [4 ounce (pack of 3)][3 piece assortment][4 ounce (pack of 2)][4 ounce (pack of 6)][4 ounce (3 count)][4 ounce (6 count)]
Smoked Bacon Sea Salt 3-Pack: Smoked Bacon Chipotle, Smoked Bacon and Onion, Smoked Peppered Bacon - All-Natural Sea Salts Slowly Smoked Over Alderwood - No Gluten, No MSG, Non-GMO (12 total oz.)
Price: \$29.99
Rating: N.A.
[Description]
[Features]
[Reviews]
[Attributes]
[Buy Now]

Action: click[4 ounce (pack of 2)]
Observation: You have clicked 4 ounce (pack of 2).

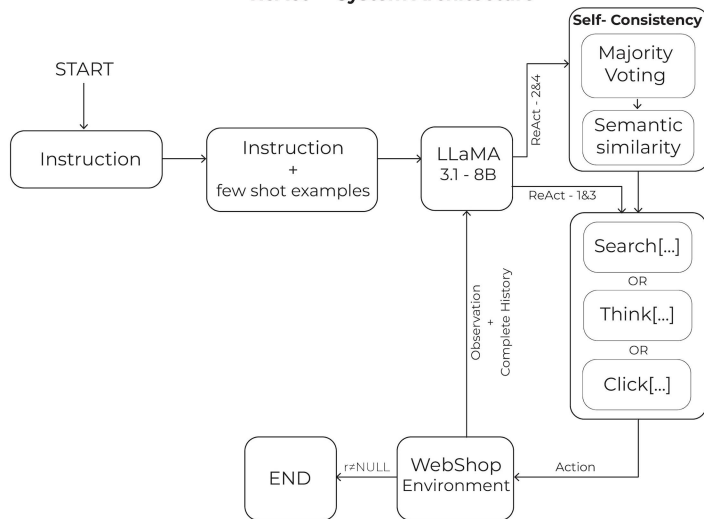
Action: click[Buy Now]
Observation: Your score (min 0.0, max 1.0): 0.75

ReAct++: Enhancing LLM Reasoning

Few-shot Prompting + Self-Consistency

System Architecture

ReAct++ System Architecture



Key Mechanisms

- **Few-shot prompting** guides the model with examples.
- Generates **multiple reasoning-action trajectories**.
- Applies **self-consistency** via majority voting and semantic similarity.

Benefits

- **Improved performance** over standard ReAct.
- **More stable decisions** through sampled consensus.

Limitations of ReAct++

Core Weaknesses

- Collapses multiple candidate actions into a **single decision**
- No notion of **confidence or uncertainty**
- Treats all sampled actions **equally**
- Relies on **heuristic aggregation** (no guarantees)
- Requires **multiple expensive LLM calls**

Decisions are brittle and inefficient

Even though ReAct++ uses multiple samples, it ultimately reduces everything to a single decision without any notion of uncertainty. This makes the decision brittle and computationally expensive.

Overall Framework

Our Framework

LLM Actions → Scoring Function → Conformal Action Set → BFS Planning → Final Action

Key Idea:

- ✓ Rank actions
- ✓ Select high-confidence set
- ✓ Explore efficiently

Our method consists of four key stages. First, the **LLM** generates candidate actions. Then we **score** them using a learned model. Using **conformal prediction**, we select a set of high-confidence actions. Finally, we explore these actions using **BFS** to find the best trajectory.

Data Generation (Ground Truth Creation)

Data Generation Pipeline

Ground-truth actions are not directly available. We construct them using a **multi-level search strategy**.

Step 1: Original Instruction

- Search using given instruction
- If success → stop

Step 2: Paraphrase Generation

- Generate 50 keywords using LLM
- Make 5 Clusters
- Select 5 representative queries

Step 3: Expanded Search

- Search using new keywords
- Explore multiple products
- If success → stop
- Else repeat step 2 & 3 in BFS Manner

Optimal action is selected from successful trajectories

Dataset Structure

Supervised data for learning action ranking

Dataset Format

Each data point contains:

- **Instruction** - task description
- **Previous Actions** - action history
- **Admissible Actions** - candidate actions
- **LLM Logits** - token-level probabilities
- **Optimal Action** - ground-truth label

Instruction:

"Find blue sneakers under \$100"

Previous Actions:

[search..., click...]

Admissible Actions:

[click[A], click[B], click[C]]

LLM Logits:

[[...], [...], [...]]

Optimal Action:

click[B]

Each data point consists of the current state, candidate actions generated by the LLM, and the optimal action obtained from our data generation pipeline. This allows us to train a model that ranks actions based on their quality.

State Representation

State Components

The state is constructed using:

- Instruction embedding
- Previous actions embedding (pooled)

Final State Vector

Concatenation of:

- Instruction embedding
- Aggregated action history

Embedding Model

- Pretrained **BERT**

We represent the state by combining the instruction and the history of previous actions. Both are embedded using BERT, and the action history is pooled before concatenation to form the final state vector.

Action Representation

Action Representation

Each action is represented using:

- Text embedding (BERT)
- LLM logits (confidence information)

Logits Processing

- Convert token log-probs into **softmax distribution**
- Discretize into **fixed bins**

Final Action Representation

- Embedding + softmax-bin features

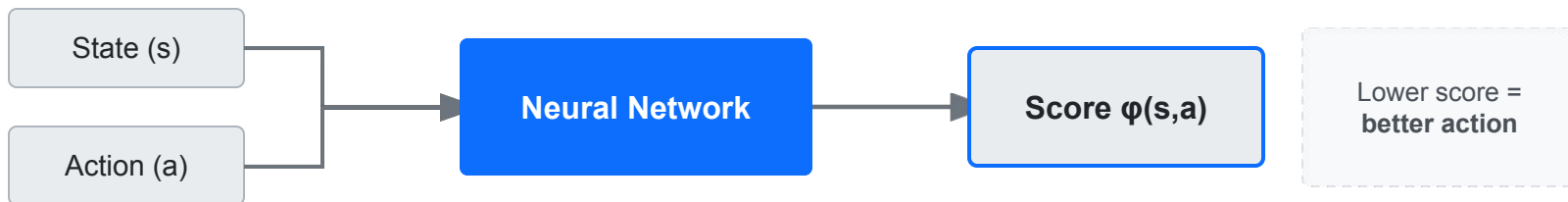
Each action is represented not only by its semantic meaning using BERT, but also by the confidence of the LLM through logits. We convert logits into softmax distributions and discretize them into bins, capturing uncertainty information.

Scoring Function

Key Idea

Learn a function that measures how well an action fits the current state

LLM outputs are stochastic and not calibrated for decision-making. A learned scoring function provides a consistent and trainable signal for ranking actions.



The scoring function maps complex state-action inputs into a single scalar value for effective ranking.

Scoring Function

Scoring Function

- Input:
 - state representation
 - action representation
- Output:
 - scalar score $\phi(s,a)$

Interpretation

- Lower score $\rightarrow$ more likely optimal
- Higher score $\rightarrow$ less relevant action

Purpose

- Rank candidate actions
- Identify high-quality actions

We learn a scoring function that takes a state and an action as input and outputs a scalar score. This score represents how well the action fits the current state, where lower values indicate better actions.

Training Objective

Training Objective

- Supervised learning using constructed dataset
- Each training example contains:
 - state representation S
 - candidate actions $\{a_1, a_2, \dots, a_k\}$
 - optimal action a^*

Approach

- Convert each (state, action) into a **binary classification problem**
- Mapping Labels:
 - Optimal action to label = 1
 - Other actions to label = 0

Learning Goal:

$$\varphi(s, a^*) < \varphi(s, a_i), \text{ for all } i \neq a^*$$

We convert the ranking problem into a binary classification problem at the action level. For each state-action pair, the optimal action is labeled as positive and others as negative. This allows the model to learn relative ordering.

Training Objective

Dataset Construction

Each state expands into training pairs: (**state**, **action**, **label**)

- label = 1 → optimal action
- label = 0 → other action

Loss Function

Binary classification task implemented via:

BCEWithLogitsLoss

Training Pipeline

Input: state (1536), action (768), softmax (10)

Output: logit → sigmoid → probability

Optimization

- Adam optimizer
- Learning rate scheduling
- Early stopping

In implementation, each state is expanded into multiple state-action pairs. The model predicts a logit for each pair, and we use BCEWithLogitsLoss to classify whether an action is optimal or not. This formulation allows efficient training and naturally supports ranking.

Motivation: Why Conformal Prediction?

But can we trust these scores?

The Problem: Model Calibration

- Neural networks are not perfectly calibrated.
- They can be overconfident or underconfident in scoring.
- Early mistakes in WebShop can lead to cascading failures.

The Solution: Statistical Safety Net

Conformal prediction provides a formal guarantee:

"Our prediction SET will contain the correct action at least $X\%$ of the time."

The scoring function's uncertainty is often unknown. By adding conformal prediction, we create a robust framework that accounts for these errors systematically.

Now let's see HOW it achieves this.

Calibration Dataset

What is it?

- A dataset used to **calibrate uncertainty estimates**
- Required for **conformal prediction guarantees**

How we construct it

- Use **validation split** (not training data)
- For each validation state (s_i):
 - Compute nonconformity scores for all actions
 - Extract score of **optimal action only**

Stored Data

$$\mathcal{D}_{cal} = \{(x_i, S_i)\}_{i=1}^n$$

x_i : state representation; S_i : nonconformity score of optimal action

Why this is important

- Ensures **no data leakage**
- Enables **statistical guarantees**: $\mathbb{P}(a^* \in C(s)) \geq 1 - \alpha$

Calibration is required for conformal prediction to provide statistical guarantees. We construct the calibration dataset from the validation split. For each state, we compute nonconformity scores and store only the score of the optimal action along with the state representation.

Conformal Prediction (Action Set Construction)

Conformal prediction allows us to construct a **set of actions** instead of selecting a single one. This set is guaranteed to contain the optimal action with high probability, based on past calibration data.

Key Idea

- Do not select a single action
- Instead, select a **set of plausible actions**
- **The threshold is NOT fixed, it adapts to each state via dual optimization (explained in following slides)**

Why this matters

- Provides **uncertainty-aware decisions**
- Avoids **overconfident single predictions**

Goal

Construct a **set of actions** $C(s)$ such that:

$$P(a^* \in C(s)) \geq 1 - \alpha$$

Procedure

For each candidate action a :

1. Compute nonconformity score S_{test}
2. Compare with calibration scores $\{S_i\}$
3. Include action if $\eta_{n+1} < 1 - \alpha$
4. (where η_{n+1} comes from dual LP)

Output

$$C(s) \subseteq \mathcal{A}$$

(prediction set)

Similar State Selection

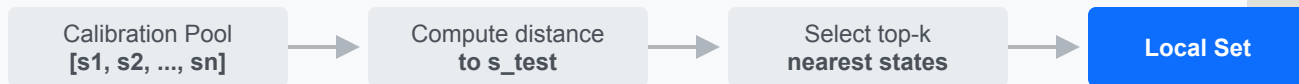
Problem with standard conformal

- Uses **global calibration dataset**
- Ignores differences between states

Our Idea: Local Calibration

- For a test state s , select **similar calibration states**

Selection Workflow



$$d_i = \|s - s_i\|^2$$

L2 distance used for similarity matching

Local calibration adapts the
threshold to the current state context

Instead of using the entire calibration dataset, we select only the states that are similar to the current test state using L2 distance. This allows us to perform local calibration, which improves accuracy and makes the method adaptive.

Example: Local Distance Computation

For our test state, we find the **most similar past states** from the calibration pool by computing the L2 distance between embeddings.

Computed Distances

Distance to State 1: 2.3 (far)

Distance to State 2: 0.8 (**close!**)

Distance to State 3: 0.6 (**very close!**)

Distance to State 4: 4.1 (very far)

Distance to State 5: 0.9 (**close!**)

Top-k Selection (k=3)

Selected local calibration states:

- **State 3:** score = 0.08, dist = 0.6
- **State 2:** score = 0.12, dist = 0.8
- **State 5:** score = 0.03, dist = 0.9

Why local calibration?

Similar states are more informative for the current context:

- **State 3:** "Find red running shoes under \$60" → High relevance to shoe-buying.
- **State 4:** "Buy a kitchen blender" → Completely different logic.

Using distant states to calibrate thresholds is noisy; local calibration is smarter.

Adaptive Threshold via Quantile Regression

Goal

- Learn a **state-dependent threshold** for nonconformity scores
- Replace fixed quantile with function $\mathbf{f}(\mathbf{x})$

Problem Setup

- Nonconformity scores: $\mathbf{S}_i$
- State features: $\mathbf{x}_i$
- Goal: learn $\mathbf{f}(\mathbf{x}) = \boldsymbol{\varphi}(\mathbf{x}_i)^\top \boldsymbol{\beta}$
- $\boldsymbol{\varphi}(\mathbf{x}_i)$ = feature representation
- $\boldsymbol{\beta}$ = parameters to learn

Objective (Quantile Regression)

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n+1} \ell_{\alpha}(S_i - f(x_i))$$

Pinball Loss

$$\ell_{\alpha}(u) = \begin{cases} \alpha \cdot u, & u < 0 \\ (1 - \alpha) \cdot u, & u \geq 0 \end{cases}$$

Key Idea

- Penalize **underestimation less**
- Penalize **overestimation more**
- Learn **quantile-based threshold**

We learn a state-dependent threshold using quantile regression, where the pinball loss penalizes errors asymmetrically to control the quantile level.

Optimization and Dual Formulation

Primal Problem

$$\min_{f \in \mathcal{F}} \sum_{i=1}^{n+1} \ell_{\alpha}(S_i - f(x_i))$$

Fit β such that threshold aligns with calibration scores and respects quantile level α .

Taking the Lagrangian dual of the primal LP gives:

Dual Formulation

- Convert to **linear program (LP)**
- Solve using dual variables η
- Dual variables η emerge from LP relaxation of primal
- Each η_i is a weight assigned to calibration state i
- η_{n+1} is the weight assigned to the test action

Decision Rule

- Compare test score with learned threshold
- Include action if:

$$S_{e_{\square}} \leq f(x_{e_{\square}})$$

Output

- Adaptive threshold
- Calibrated prediction set

This is the PRIMAL formulation motivation only. We do NOT train a separate model. Instead, we solve the DUAL form of this problem directly at inference time.

Adaptive Threshold via Dual Optimization

Instead of using a fixed threshold, we compute a **state-dependent threshold** using a dual optimization problem. This enables more precise, adaptive decision-making by comparing test and calibration scores under constraints.

Motivation

- Standard conformal uses **fixed (global) thresholds**
- Not suitable for **state-dependent uncertainty**

Our Approach

Compute a **state-dependent threshold** by solving a dual optimization problem that adjusts to the local context of the test state.

Mathematical Framework

Formulation

$$\max_{\eta} \sum_{i=1}^n \eta_i S_i + \eta_{n+1} S_{test}$$

Constraints

$$\begin{aligned} -\alpha &\leq \eta_i \leq 1 - \alpha \\ \Phi^T \eta &= 0 \end{aligned}$$

Decision Rule

$$a \in C(s) \quad \text{if} \quad \eta_{n+1} < 1 - \alpha$$

Adaptive Threshold Algorithm

Inputs

Test action score:

S_{test}

Calibration score :

$\{S_i\}_{i=1}^n$

SVD-projected State features:

$\Phi \in \mathbb{R}^{(k+1) \times d}$

Miscoverage level:

α

Algorithm Steps

Step 1: Select Relevant Calibration Scores

Choose top-k similar states and extract scores $\{S_i\}$ and apply SVD to build $\Phi \in \mathbb{R}^{(k+1) \times d}$.

Step 2: Solve Dual Optimiz: $\max_{\eta} \sum_{i=1}^n \eta_i S_i + \eta_{n+1} S_{test}$

$$-\alpha \leq \eta_i \leq 1 - \alpha$$

$$\Phi^T \eta = 0$$

Step 3: Constraints

Step 4: Compute Test Weight

Obtain η_{n+1} from the solution.

Step 5: Apply Decision Rule $a \in C(s) \quad \text{if} \quad \eta_{n+1} < 1 - \alpha$

Include action if $\eta_{n+1} < 1 - \alpha$.

Output

- Include / Exclude decision
- Forms final prediction set $C(s)$

Computes a state-dependent threshold by comparing the test action score with similar calibration scores and uses dual optimization to enforce statistical constraints along with state similarity. The value η_{n+1} indicates how well the action aligns with past data, and actions are included in the prediction set only if they satisfy the adaptive confidence condition.

Prediction Set Construction

Goal

Construct the final set of high-confidence actions:

$$C(s) = \{a \in A \mid \eta_{n+1}(a) < 1 - \alpha\}$$

Procedure (per action)

1. Compute score $\phi(s,a)$
2. Select similar calibration states
3. Project features (SVD)
4. Solve dual optimization
5. Apply inclusion rule

Output

- Final action set $C(s)$
- Actions sorted by nonconformity

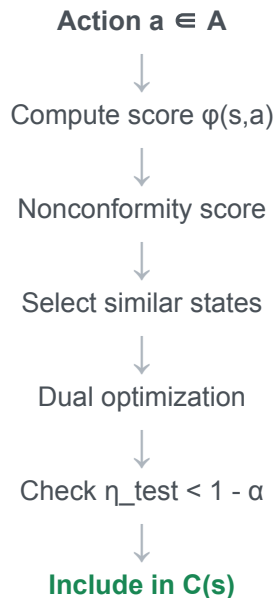
Key Property

- **High-confidence** actions only
- Size adapts based on uncertainty

$\eta_{n+1}(a)$ is computed separately for each candidate action a by solving the dual LP with $S_{\text{test}} = \text{Nonconformity}(s, a)$

Prediction Set Construction

Algorithmic Flow



Process Summary

For each candidate action, we compute its score and nonconformity, compare it with similar calibration states using dual optimization, and include it in the prediction set if it satisfies the threshold condition.

Final Outcome

Set of high-confidence actions.

Full Pipeline: Uncertainty-Aware Action Selection

1. LLM Generation

Generate multiple candidate actions

2. Scoring Function

Evaluate each action

3. Nonconformity

Convert scores to uncertainty measure

4. State Selection

Select relevant calibration states

5. Adaptive Threshold

Compute state-dependent threshold

6. Prediction Set

Select high-confidence actions

7. BFS Exploration

Explore action set efficiently

Output

- Final action / trajectory
- Maximizes success with uncertainty awareness

Our system starts by generating candidate actions using the LLM, then evaluates them using a scoring function. We convert these scores into nonconformity measures and use conformal prediction with adaptive thresholds to construct a set of high-confidence actions. Finally, we explore this set using BFS to find the best trajectory.

Example

Task: "I need blue running shoes under \$50" | **Previous Action:** `search[blue running shoes]`

Action	Nonconformity Score	Meaning
A1 - click[Nike \$45]	0.04	Very likely optimal
A2 - click[Adidas \$55]	0.82	Very unlikely optimal
A3 - click[Next Page]	0.48	Uncertain
A4 - click[Back to Search]	0.88	Very unlikely optimal

Key Insight: Low S = good action. High S = bad action.

Picking **a1** purely based on score is risky. We need **conformal prediction** to handle model uncertainty.

Similar State Selection & Building Φ

Calibration Pool (built from validation data before gameplay):

Cal. State	Task Description	Optimal Action Score S_i
X_1	"Find red sneakers under \$60"	0.08
X_2	"Buy blue sports shoes \$80"	0.12
X_3	"Get running shoes \$45"	0.03
X_4	"Buy kitchen blender"	0.45

Step: Find top-k similar states using L2 distance:

Cal. State	Distance d_i	Selected?
X_1 - red sneakers \$60	0.6	Yes
X_2 - blue sports shoes	0.8	Yes
X_3 - running shoes \$45	0.9	Yes
X_4 - kitchen blender	4.1	No - irrelevant

SVD Projection $\rightarrow$ Build Φ

Raw embeddings are 1536-dim $\rightarrow$ LP would have 1536 constraints but only 4 variables $\rightarrow$ infeasible.

SVD reduces selected states to $d = 2$ dimensions:

3 cal. states $[3 \times 1536] \rightarrow$ SVD $\rightarrow V [1536 \times 2] \rightarrow$ projection $\rightarrow \Phi [4 \times 2]$

Dual LP: Building the Prediction Set C(s)

For each candidate action, solve this LP with $S_{\text{test}} = S(s,a)$:

$$\max_{\eta} \sum_{i=1}^n \eta_i S_i + \eta_{n+1} S_{\text{test}}$$

Subject to: -

$$-\alpha \leq \eta_i \leq 1 - \alpha$$

$$\Phi^T \eta = 0$$

Notation & Constraints

- η_i : Weight assigned to calibration state i
- $\eta_{\{k+1\}}$: Weight assigned to **test action**
- S_i : Calibration optimal scores [0.08, 0.12, 0.03]
- S_{test} : Nonconformity of candidate action
- $\alpha = 0.1 \rightarrow$ bounds are [-0.1, 0.9]
- $\Phi^T \eta = 0$: Links test score to calibration neighborhood geometry.

Testing a1 (Nike \$45, $S_{\text{test}} = 0.04$)

Max: $\eta_1 \times 0.08 + \eta_2 \times 0.12 + \eta_3 \times 0.03 + \eta_4 \times 0.04$

Low S_{test} keeps η_4 balanced.

$\eta_4 = 0.2 < 0.9$  $\rightarrow$ **INCLUDED**

Testing a2 (Adidas \$55, $S_{\text{test}} = 0.82$)

Max: $\eta_1 \times 0.08 + \eta_2 \times 0.12 + \eta_3 \times 0.03 + \eta_4 \times 0.82$

High coefficient pushes η_4 to upper bound.

$\eta_4 = 0.9 < 0.9$? NO  $\rightarrow$ **EXCLUDED**

Conformal Action Set Construction Results

Action	S_test	$\eta_{\square+1}$	$< 1-\alpha?$	In C(s)?
a1 Nike \$45	0.04	0.2	yes	INCLUDED
a2 Adidas \$55	0.82	0.9	no	EXCLUDED
a3 Next Page	0.48	0.5	yes	INCLUDED
a4 Back Search	0.88	0.9	no	EXCLUDED

Final Prediction Set: $C(s) = \{a_1, a_3\}$

$$P(a \in C(s)) \geq 1 - \alpha = 0.90$$

BFS now explores **ONLY a1 and a3** — with 90% statistical confidence that the optimal action is in this set.

BFS Planning over Conformal Action Sets

Motivation

- Conformal prediction returns **multiple valid actions**
- Need to **explore them systematically**

Core Idea

- Use **Breadth-First Search (BFS)**
- Explore all actions in prediction set **level-by-level**

Node Representation

Each node stores:

- Env state (session snapshot)
- Previous actions
- Full LLM prompt
- Instruction

Key Steps (per node)

1. Restore environment state
2. Generate actions (LLM)
3. Separate: *think[]* and *click[]*
4. Compute conformal set
5. Expand BFS with selection

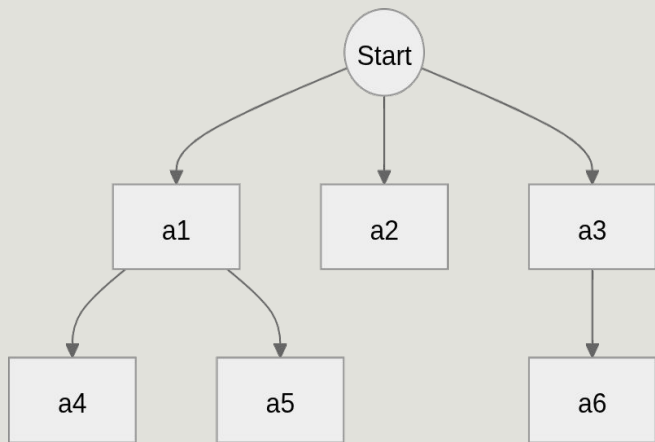
Stopping Criteria

- **reward = 1.0**
Stop immediately
- **Partial reward**
Continue search
- **Budget/depth limit**
Return best trajectory

BFS Exploration & Search Complexity

Systematic Exploration

After constructing the conformal action set, we use BFS to explore all high-confidence actions systematically. Each node stores the full environment state, allowing efficient branching without recomputation.

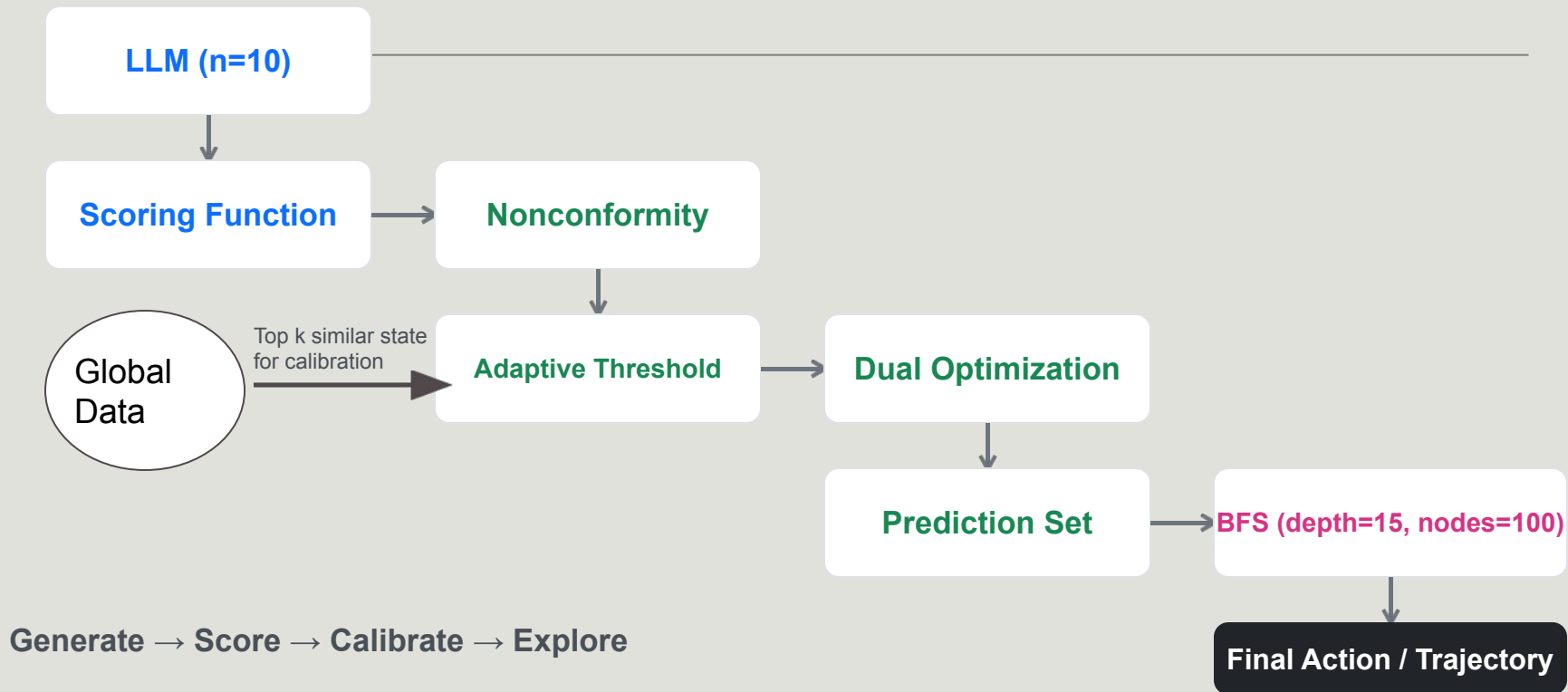


Parameter	Value	Meaning
Max Depth	15	Max steps per trajectory
Node Budget	100	Max BFS nodes explored
n_samples	10	LLM candidates per node
k (Top-k)	50	Similar calibration states

Search Complexity Control

We limit BFS using a maximum depth of 15 and a node budget of 100 to control search complexity. At each node, we sample 10 candidate actions from the LLM and use the top 50 similar calibration states for conformal prediction.

End-to-End Pipeline



Experimental Setup

Environment

- WebShop benchmark
- 200 instructions

Model

- Qwen 2.5 - 3B
- ReAct++ baseline

Our Method

Scoring + Conformal
Prediction + BFS

Evaluation Metrics

- Success Rate (SR)
- Average Reward
- Trajectory Length

We evaluate our method on the WebShop environment with 200 instructions. We compare against ReAct++ baselines using success rate, reward, and trajectory efficiency.

Main Results

Performance

Success Rate: 52.5%

Average Reward: 0.78

Episodes: 200

Key Statistics

Avg Length: 4.045

Min: 0

Max: 13

Key Insight

- Significant improvement over ReAct++
- More stable performance

Insight

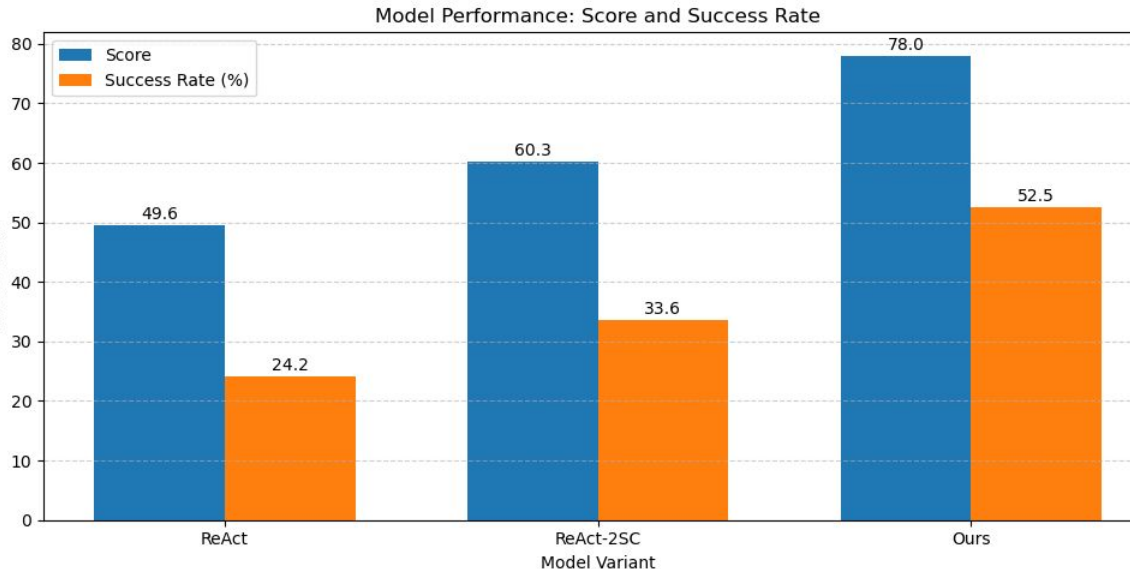
- Majority concentrated at **3, 4, 5 steps**
- Balanced exploration

Comparison with ReAct++

Key Takeaway

Our method improves success rate from **33.6%** to **52.5%**.

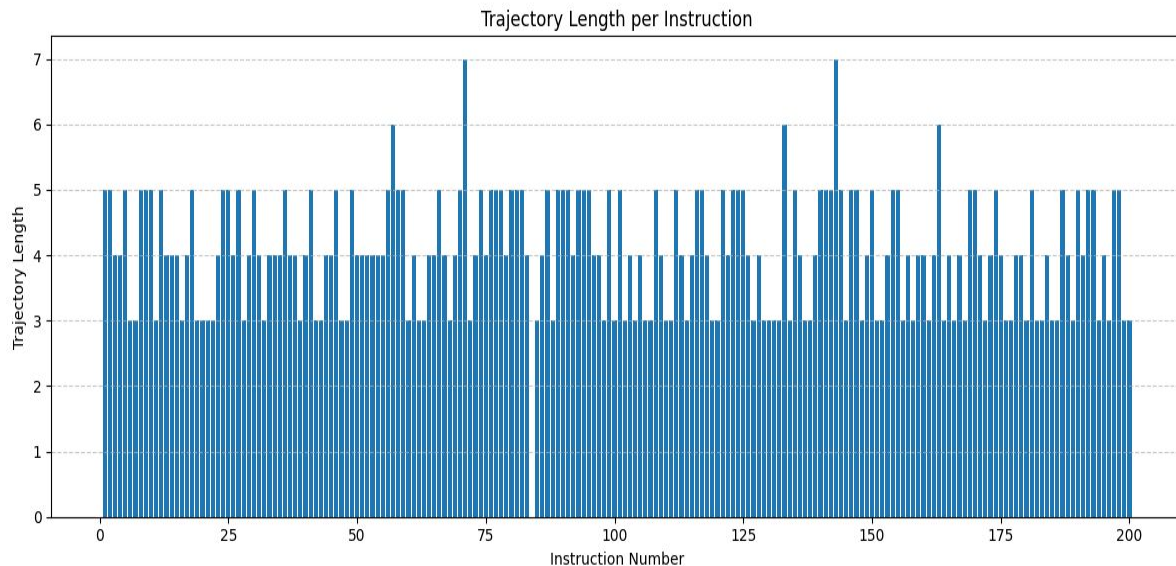
This demonstrates the effectiveness of *uncertainty-aware action selection*.



Trajectory Length per Instruction

Observations

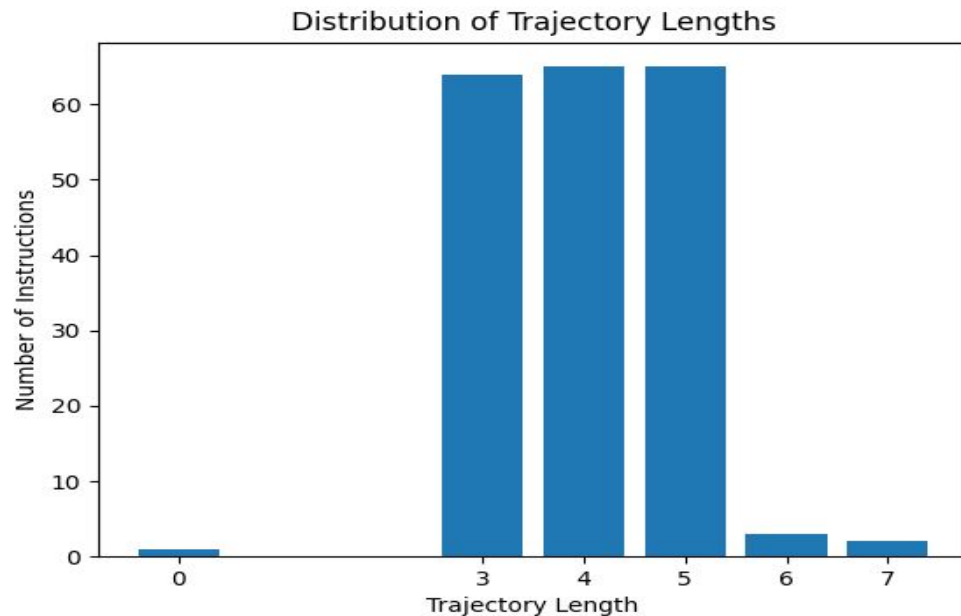
- Most trajectories: **3–5 steps**
- Few long trajectories



Distribution of Trajectory Lengths

Observations

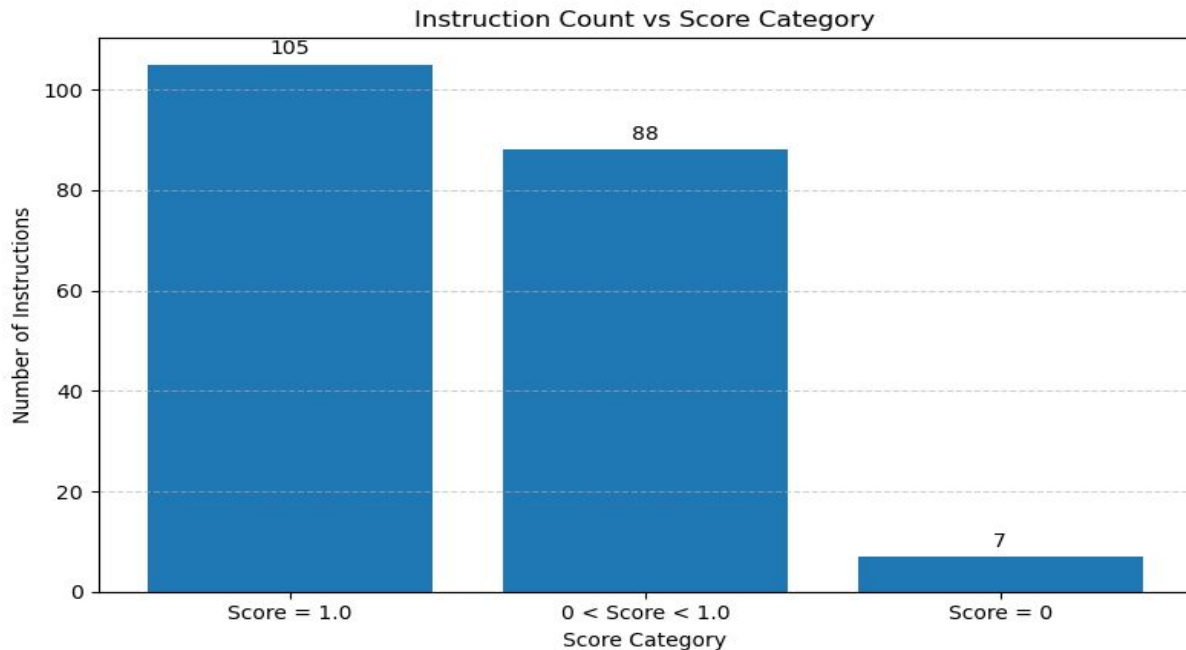
- Majority concentrated in **3 - 5 steps**
- Balanced exploration



Score Analysis

Observations

- Over **50%** of tasks achieved a perfect score (1.0)
- High reliability in our proposed approach
- Minimal failure cases (Score = 0)



Scoring Function Performance (Recall@K)

Key Observations

- **Best performance** at 100 epochs
- Slight drop at 150 epochs indicates **overfitting**
- High Recall@K confirms **strong ranking** ability

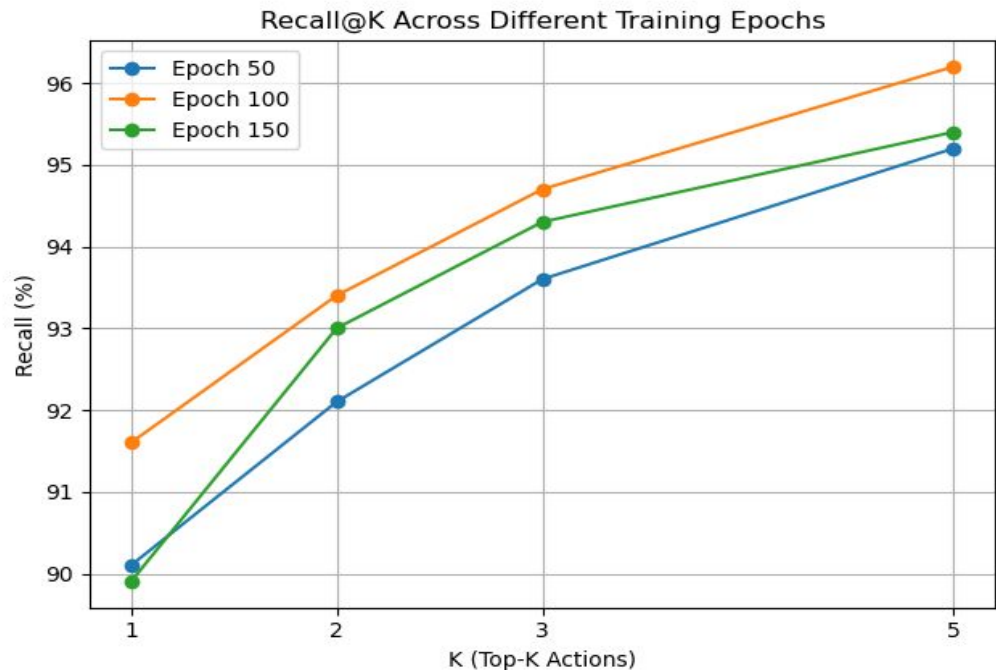
Epochs	R@1	R@2	R@3	R@5
50	90.1	92.1	93.6	95.2
100	91.6	93.4	94.7	96.2
150	89.9	93.0	94.3	94.4

We observe that performance improves up to 100 epochs and then slightly degrades, indicating mild overfitting. The high Recall@K values confirm that the scoring function reliably ranks the optimal action among top candidates.

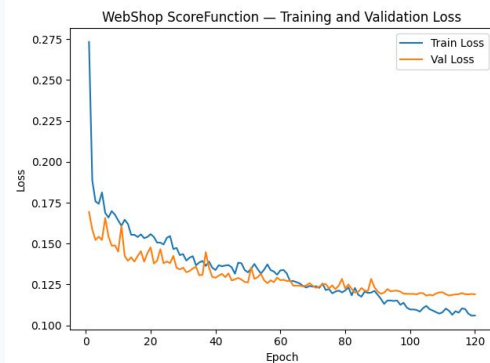
Recall vs Epoch Analysis

Key Observations

- Recall improves significantly from **50** → **100 epochs**.
- A slight performance drop is noted at **150 epochs**, indicating potential overfitting.
- Consistent growth across all K values (Top-K actions).



Training Dynamics: Loss vs. Epoch Analysis



Key Insight

The training process shows consistent convergence across different configurations. A sharp initial decrease in loss is followed by stable refinement, ensuring the scoring function generalizes effectively to unseen task environments.

Key Insights

Uncertainty-aware Selection

Conformal prediction filters unreliable candidates, significantly reducing incorrect actions.

Set-based Decisions

Moving beyond single-action selection to multiple candidates improves robustness.

Adaptive Thresholding

Local calibration via state-dependent thresholds is critical for decision quality.

Efficient Exploration (BFS)

Breadth-First Search finds superior trajectories compared to standard greedy methods.

Reliable Scoring Function

A high Recall@K metric ensures correct actions are consistently ranked and considered.

Why It Works

Better Action Selection

Scoring function ranks actions effectively. High Recall@K ensures correct actions are consistently considered.

Uncertainty Awareness

Conformal prediction filters unreliable actions, significantly reducing incorrect decisions.

Adaptive Decision Making

State-dependent thresholds enable handling varying difficulty across diverse tasks.

Efficient Exploration

BFS explores multiple high-confidence actions, avoiding the pitfalls of greedy failures.

Conclusion

Summary of Work

LLM agents face key challenges:

- Stochastic action generation
- Lack of reliability in decision-making
- Inefficient exploration

Unified Framework Proposal

- **State-action scoring:** evaluates compatibility between state and actions
- **Conformal prediction:** provides uncertainty-aware action selection
- **Adaptive thresholding:** enables state-dependent decision making
- **BFS-based exploration:** systematically explores high-confidence actions

WebShop Evaluation (200 tasks): Achieving significant improvements in performance through integrated learning and search.

Key Contributions

Uncertainty-Aware Action Selection

- Formulated as a **conformal prediction** problem
- Enables **set-valued decisions** with statistical guarantees

State-Action Scoring Function

- Learns to **rank candidate actions** effectively
- Achieves strong **Recall@K** performance

Adaptive Thresholding

- Uses **dual optimization** for state-dependent thresholds
- Improves calibration over fixed global methods

BFS over Conformal Action Sets

- Explores multiple **high-confidence actions** efficiently
- Improves trajectory quality over greedy approaches

End-to-End Integrated Pipeline

- Synergizes **learning + uncertainty + search**
- Strong empirical results in the **WebShop** environment

Future Work & Final Takeaway

Future Work

- **Extend framework to:**
 - Other agentic environments (tool use, embodied tasks)
- **Improve efficiency:**
 - Reduce BFS search cost
 - Optimize conformal computation
- **Enhance scoring models:**
 - Use stronger LLM-based or hybrid architectures

Final Takeaway

- Uncertainty-aware methods significantly improve reliability
- Set-based decision making is more robust than single-action selection
- Combining **learning + calibration + search** is key for LLM agents

Limitations

Computational Cost

- BFS exploration increases runtime
- Dual optimization adds overhead

Dependence on Calibration Quality

- Performance depends on quality of calibration dataset
- Distribution mismatch may affect guarantees

Environment Specificity

- Trained on WebShop
- May require adaptation for other domains

Approximate Conditional Guarantees

- Local calibration is approximate

While our method improves performance, it introduces additional computational cost and depends on calibration quality. Extending it to other environments and improving efficiency are important directions for future work.

References

Core Research Papers

Yao et al., 2023 — *ReAct: Synergizing Reasoning and Acting in Language Models*

arXiv preprint arXiv:2210.03629

Yao et al., 2022 — *WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents*

arXiv preprint arXiv:2207.01206

Wang et al., 2022 — *Self-Consistency Improves Chain-of-Thought Reasoning in Language Models*

arXiv preprint arXiv:2203.11171

Angelopoulos & Bates, 2021 — *A Gentle Introduction to Conformal Prediction*

arXiv preprint arXiv:2107.07511

Gibbs et al., 2023 — *Conformal Prediction with Conditional Guarantees*

arXiv preprint arXiv:2305.12616

Cherian et al., 2024 — *Large Language Model Validity via Enhanced Conformal Prediction Methods*

arXiv preprint arXiv:2406.09714

Thank You!

Questions and Discussion

